



Object Oriented Python Programming

Unit 2 — Advanced OOP Concepts

Topics Covered

- ◆ Inheritance
- ◆ Method Overriding
- ◆ Polymorphism
- ◆ Special (Magic / Dunder) Methods
- ◆ Exception Handling using try and except

ETHANS
CLOUD & AI ACADEMY



Chapter 1

Inheritance

In the real world, children often inherit traits from their parents — appearance, habits, or even skills. Python allows something very similar in programming. When you design classes (blueprints for objects), you can create a new class that automatically gets everything from an existing class. This concept is called Inheritance.

1.1 What is Inheritance?

Inheritance	A feature of Object-Oriented Programming that allows a new class (called a Child class or Subclass) to automatically receive all the attributes and methods of an existing class (called a Parent class or Superclass), without rewriting them.
-------------	---

 **Real-World Analogy — A Company Hierarchy**

Think of a company with a general 'Employee' profile.
Every employee has a name, employee ID, and a method called `check_in()`.

Now, a 'Manager' is also an employee — but with additional responsibilities like approving leaves.
A 'Intern' is also an employee — but with a limited access level.

Instead of writing name, employee ID, and `check_in()` again for Manager and Intern, you simply inherit all of that from Employee and only add what is new.

Why Use Inheritance?

Inheritance solves one of the most common problems in software development — code repetition. Without inheritance, if you need the same attributes or behaviour across multiple classes, you would have to copy and paste code into each class. This makes maintenance painful — if you fix a bug in one place, you have to fix it everywhere.

Without Inheritance	With Inheritance
Copy the same attributes into every class	Write attributes once in the Parent class
If a method changes, update it in every class	Update in the Parent — all children get the fix automatically
Code is long and difficult to read	Code is short, clean, and organized
High chance of inconsistency between classes	All child classes behave consistently

1.2 Syntax of Inheritance

To create a child class that inherits from a parent class, you write the parent class name inside parentheses when defining the child class.

Python Code

```
# Syntax of Inheritance

class ParentClass:
    # Attributes and methods of the parent
    pass

class ChildClass(ParentClass): # ChildClass inherits from ParentClass
    # Additional attributes and methods for the child
    pass
```

Figure 1.1 — Basic syntax of Inheritance



Key Terminology

Parent Class (also called Base Class or Superclass): The existing class whose features are being reused.

Child Class (also called Derived Class or Subclass): The new class that inherits from the Parent.

The child class automatically gets all attributes and methods of the parent class.

1.3 Your First Inheritance Example

Let us start with a practical, professional example. We will build a small system for a company that manages different types of employees.

Step 1 — Define the Parent Class

The parent class Employee contains everything common to all employees.

Python Code

```
# Parent Class
class Employee:

    def __init__(self, name, employee_id, department):
        self.name = name
        self.employee_id = employee_id
        self.department = department
```

```
def display_info(self):
    print('--- Employee Record ---')
    print('Name      : ', self.name)
    print('Employee ID : ', self.employee_id)
    print('Department  : ', self.department)

def check_in(self):
    print(self.name, 'has checked in.')
```

Figure 1.2 — The Employee parent class

Step 2 — Create a Child Class (Manager)

The Manager class inherits from Employee. It automatically gets name, employee_id, department, display_info(), and check_in(). We only add what is unique to a Manager.

Python Code

```
# Child Class — inherits from Employee
class Manager(Employee):

    def __init__(self, name, employee_id, department, team_size):
        super().__init__(name, employee_id, department) # Call parent's __init__
        self.team_size = team_size                      # Extra attribute

    def approve_leave(self, employee_name):
        print(self.name, 'has approved leave for', employee_name) # Extra method
```

Figure 1.3 — The Manager child class

What is super()?

super() is a built-in Python function that gives you access to the parent class.
super().__init__(...) calls the parent class's constructor so that the inherited attributes (name, employee_id, department) are properly initialized.
Without super(), the child class would not set up the parent's attributes.

Step 3 — Create Another Child Class (Intern)

Python Code

```
# Another Child Class
class Intern(Employee):

    def __init__(self, name, employee_id, department, internship_duration):
        super().__init__(name, employee_id, department)
```

```

self.internship_duration = internship_duration

def submit_report(self):
    print(self.name, 'has submitted the internship report.')

```

Figure 1.4 — The Intern child class

Step 4 — Use All Three Classes

Python Code

```

# Creating objects
emp1 = Employee('Ravi Mehta', 'EMP001', 'Finance')
mgr1 = Manager('Priya Singh', 'MGR001', 'Engineering', 12)
int1 = Intern('Aakash Patel', 'INT001', 'Marketing', '6 months')

# Using inherited methods on child objects
emp1.display_info()
emp1.check_in()

print()

mgr1.display_info()    # Inherited from Employee
mgr1.check_in()        # Inherited from Employee
mgr1.approve_leave('Ravi Mehta') # Manager's own method

print()

int1.display_info()    # Inherited from Employee
int1.submit_report()   # Intern's own method

```

Figure 1.5 — Using inherited classes

Output

```

--- Employee Record ---
Name      : Ravi Mehta
Employee ID : EMP001
Department : Finance
Ravi Mehta has checked in.

--- Employee Record ---
Name      : Priya Singh
Employee ID : MGR001
Department : Engineering
Priya Singh has checked in.
Priya Singh has approved leave for Ravi Mehta

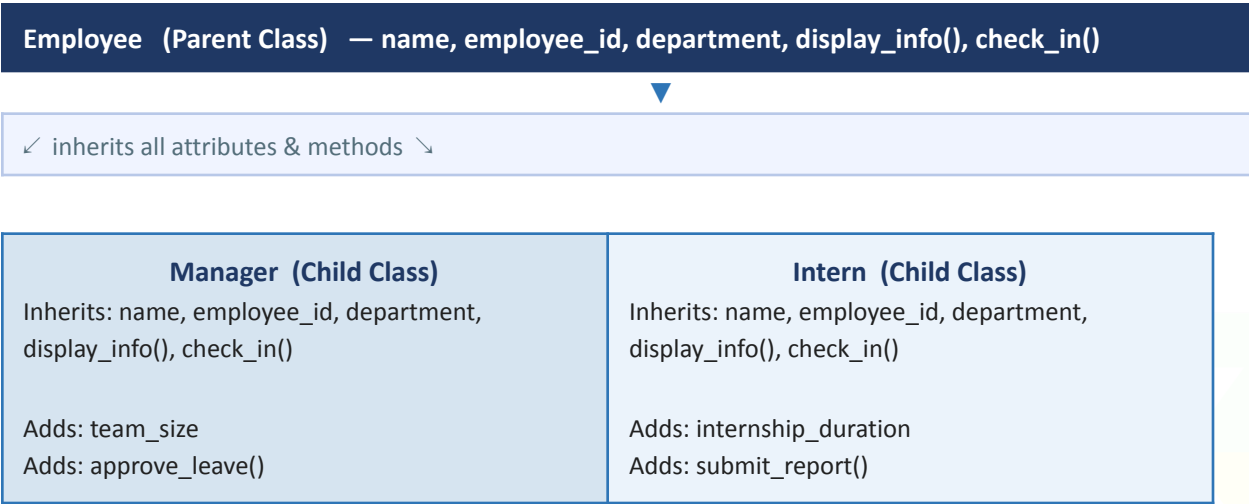
--- Employee Record ---

```

```
Name      : Aakash Patel
Employee ID : INT001
Department  : Marketing
Aakash Patel has submitted the internship report.
```

1.4 The Inheritance Hierarchy (Visual)

It helps to visualize inheritance as a tree or hierarchy. The parent class is at the top, and child classes branch out below it.



CLOUD & AI ACADEMY

1.5 Types of Inheritance

Python supports multiple types of inheritance. Understanding these helps you design better class structures.

Type	Description	Example
Single Inheritance	One child class inherits from one parent class.	Manager inherits from Employee
Multi-level Inheritance	A child class inherits from a parent, which itself inherits from another class.	Intern → Employee → Person
Multiple Inheritance	A child class inherits from more than one parent class.	class C(A, B)
Hierarchical Inheritance	Multiple child classes inherit from the same parent class.	Manager, Intern both from Employee

Multi-Level Inheritance Example

In multi-level inheritance, a chain is formed. Class B inherits from A, and Class C inherits from B — so C gets everything from both A and B.

Python Code

```
# Multi-Level Inheritance

class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def introduce(self):
        print('Hi, I am', self.name, 'and I am', self.age, 'years old.')

class Employee(Person):      # Employee inherits from Person
    def __init__(self, name, age, company):
        super().__init__(name, age)
        self.company = company
    def work(self):
        print(self.name, 'works at', self.company)

class Manager(Employee):     # Manager inherits from Employee (and transitively Person)
    def __init__(self, name, age, company, team_size):
        super().__init__(name, age, company)
        self.team_size = team_size
    def manage(self):
        print(self.name, 'manages a team of', self.team_size, 'people.')

mgr = Manager('Priya Singh', 35, 'TechCorp', 15)
mgr.introduce() # From Person
mgr.work()      # From Employee
mgr.manage()    # From Manager
```

Figure 1.7 — Multi-level inheritance: Person → Employee → Manager

Output

```
Hi, I am Priya Singh and I am 35 years old.
Priya Singh works at TechCorp
Priya Singh manages a team of 15 people.
```

1.6 Checking Inheritance — `isinstance()` and `issubclass()`

Python provides two built-in functions that let you inspect inheritance relationships at runtime.

Python Code

```
# isinstance() — checks if an object belongs to a class (or its parent chain)
print(isinstance(mgr, Manager)) # True — mgr is a Manager
print(isinstance(mgr, Employee)) # True — Manager inherits from Employee
print(isinstance(mgr, Person)) # True — transitively inherits from Person

# issubclass() — checks if one class is derived from another
print(issubclass(Manager, Employee)) # True
print(issubclass(Manager, Person)) # True
print(issubclass(Employee, Manager)) # False — reverse is not true
```

Figure 1.8 — *isinstance()* and *issubclass()* functions

Output

```
True
True
True
True
True
False
```

ETIHANS
CLOUD & AI ACADEMY



Chapter 2

Method Overriding

Inheritance allows child classes to reuse methods from a parent class. But what happens when the inherited method does not quite fit the child class? For example, all employees have a `display_info()` method — but a Manager might want to display the team size too. Method Overriding is the solution.

2.1 What is Method Overriding?

Method Overriding

When a child class defines a method with the exact same name as a method in its parent class, the child's version replaces (overrides) the parent's version. The child class provides its own specific implementation for that method.



Real-World Analogy — Standard Operating Procedure

A company has a standard procedure for sending emails — all employees follow it.

However, the Legal department has additional compliance steps they must include in every email.

The Legal team 'overrides' the standard email procedure with their own version.

The standard procedure still exists for everyone else — but Legal uses their customized one.

2.2 How Method Overriding Works

To override a method, simply define a method in the child class with the same name as the one in the parent class. Python will always call the child's version when the method is called on a child object.

Python Code

Method Overriding — Basic Example

```
class Employee:
    def __init__(self, name, employee_id):
        self.name = name
        self.employee_id = employee_id

    def display_info(self):
        print('--- Employee ---')
        print('Name      : ', self.name)
```

```
print('Employee ID :', self.employee_id)

class Manager(Employee):
    def __init__(self, name, employee_id, team_size):
        super().__init__(name, employee_id)
        self.team_size = team_size

    def display_info(self):      # Overrides Employee's display_info()
        print('--- Manager ---')
        print('Name      :', self.name)
        print('Employee ID :', self.employee_id)
        print('Team Size  :', self.team_size) # Extra info for Manager

emp = Employee('Ravi Mehta', 'EMP001')
mgr = Manager('Priya Singh', 'MGR001', 12)

emp.display_info() # Calls Employee's display_info()
print()
mgr.display_info() # Calls Manager's overridden display_info()
```

Figure 2.1 — Method overriding in action

Output

```
--- Employee ---
Name      : Ravi Mehta
Employee ID : EMP001

--- Manager ---
Name      : Priya Singh
Employee ID : MGR001
Team Size  : 12
```

2.3 Using super() in Overriding — Extending the Parent

Sometimes you do not want to completely replace the parent method — you want to keep the parent's behaviour and add to it. You can use `super()` to call the parent's method first, then add extra steps in the child.

Real-World Analogy — Extending a Template

A company has a standard email sign-off template that everyone uses.
The CEO uses the standard sign-off, but also adds a company tagline below it.

The CEO does not write a completely new sign-off — they extend the existing one.
This is exactly what `super()` does in method overriding.

Python Code

```
class Employee:
    def display_info(self):
        print('Name      : ', self.name)
        print('Employee ID : ', self.employee_id)
        print('Department : ', self.department)

class Manager(Employee):
    def display_info(self):
        super().display_info()      # Calls Employee's display_info() first
        print('Team Size  : ', self.team_size) # Then adds Manager-specific info
        print('Can Approve : ', 'Yes')

mgr = Manager('Priya Singh', 'MGR001', 'Engineering', 12)
mgr.display_info()
```

Figure 2.2 — Using super() to extend a parent method

Output

```
Name      : Priya Singh
Employee ID : MGR001
Department : Engineering
Team Size  : 12
Can Approve : Yes
```

2.4 A Complete Professional Example — Bank Account System

Let us build a more complete example to see method overriding in a realistic scenario. We will design a basic bank account system where different account types have different interest calculation methods.

Python Code

```
class BankAccount:
    def __init__(self, account_holder, account_number, balance):
        self.account_holder = account_holder
        self.account_number = account_number
        self.balance = balance

    def display_account(self):
        print('Account Holder : ', self.account_holder)
        print('Account Number : ', self.account_number)
        print('Balance      : Rs.', self.balance)

    def calculate_interest(self):
```

```

        interest = self.balance * 0.03 # 3% interest — standard rate
        print('Standard Interest (3%) : Rs.', interest)
        return interest

class SavingsAccount(BankAccount):
    def calculate_interest(self): # Overrides BankAccount's method
        interest = self.balance * 0.06 # 6% interest for savings
        print('Savings Interest (6%) : Rs.', interest)
        return interest

class FixedDepositAccount(BankAccount):
    def __init__(self, account_holder, account_number, balance, duration_years):
        super().__init__(account_holder, account_number, balance)
        self.duration_years = duration_years

    def calculate_interest(self): # Overrides BankAccount's method
        interest = self.balance * 0.085 * self.duration_years # 8.5% per year
        print('FD Interest (8.5% x', self.duration_years, 'years) : Rs.', interest)
        return interest

# Creating objects
basic = BankAccount('Rohan Shah', 'ACC001', 50000)
saving = SavingsAccount('Anita Joshi', 'SAV002', 80000)
fd = FixedDepositAccount('Vikram Nair', 'FD003', 200000, 3)

print('=== Basic Account ===')
basic.display_account()
basic.calculate_interest()

print()
print('=== Savings Account ===')
saving.display_account()
saving.calculate_interest()

print()
print('=== Fixed Deposit Account ===')
fd.display_account()
fd.calculate_interest()

```

Figure 2.3 — Method overriding in a Bank Account System

Output

```

=== Basic Account ===
Account Holder : Rohan Shah
Account Number : ACC001
Balance       : Rs. 50000
Standard Interest (3%) : Rs. 1500.0

=== Savings Account ===

```

```
Account Holder : Anita Joshi
Account Number : SAV002
Balance      : Rs. 80000
Savings Interest (6%) : Rs. 4800.0

=== Fixed Deposit Account ===
Account Holder : Vikram Nair
Account Number : FD003
Balance      : Rs. 200000
FD Interest (8.5% x 3 years) : Rs. 51000.0
```

2.5 How Python Decides Which Method to Call — MRO

When you call a method on an object, Python follows a specific search order to find which version of the method to use. This order is called the Method Resolution Order (MRO).

Method Resolution Order (MRO) — Search Path

- Step 1: Python looks in the child class first.
- Step 2: If not found, it looks in the parent class.
- Step 3: If not found, it looks further up the chain.
- Step 4: If still not found, Python raises an `AttributeError`.

The child's version always takes priority over the parent's version.

Python Code

```
# Viewing the MRO of a class
print(Manager.__mro__)
# Output: (<class 'Manager'>, <class 'Employee'>, <class 'object'>)

# Or use .mro() method:
print(Manager.mro())
```

Figure 2.4 — Viewing the Method Resolution Order



Note — All Classes Inherit from 'object'

In Python 3, every class automatically inherits from a built-in base class called 'object'. This is why you see 'object' at the end of every MRO chain. The 'object' class provides some basic default behaviours to all Python classes.

Chapter 3

Polymorphism

The word 'Polymorphism' comes from Greek — poly (many) + morphe (forms). In programming, it means that a single action can behave differently depending on the object it is applied to. This is one of the most powerful and elegant ideas in Object-Oriented Programming.

3.1 What is Polymorphism?

Polymorphism

The ability of different objects to respond to the same method name (or operator) in their own unique way. The same interface — the same method call — produces different behaviour depending on the object type.

**Real-World Analogy — The 'Submit' Button**

Think of a 'Submit' button on a website.

On a Login page — Submit logs the user in.

On a Registration page — Submit creates a new account.

On a Payment page — Submit processes the payment.

The button label says 'Submit' in all three cases — same name, same action — but each page handles it differently. That is polymorphism.

3.2 Polymorphism Through Method Overriding

The most common form of polymorphism in Python comes from method overriding (which we studied in Chapter 2). When different child classes all override the same parent method, you get polymorphic behaviour — each class responds to the same method call in its own way.

Python Code

```
# Polymorphism via Method Overriding
```

```
class Notification:
```

```
    def __init__(self, recipient, message):
```

```
        self.recipient = recipient
```

```
        self.message = message
```

```
    def send(self):          # Generic send — to be overridden
```

```
print('Sending notification to', self.recipient)

class EmailNotification(Notification):
    def send(self):          # Overrides Notification.send()
        print('Sending EMAIL to', self.recipient, ':', self.message)

class SMSNotification(Notification):
    def send(self):          # Overrides Notification.send()
        print('Sending SMS to', self.recipient, ':', self.message)

class PushNotification(Notification):
    def send(self):          # Overrides Notification.send()
        print('Sending PUSH to', self.recipient, 'via app :', self.message)

# Create a list of different notification objects
notifications = [
    EmailNotification('alice@example.com', 'Your order has been shipped.'),
    SMSNotification(' +91-9876543210', 'OTP: 482910'),
    PushNotification('user_alice', 'You have a new message.'),
]

# Call the same method on each — polymorphism in action
for notification in notifications:
    notification.send()
```

Figure 3.1 — Polymorphism through method overriding

Output

```
Sending EMAIL to alice@example.com : Your order has been shipped.
Sending SMS to +91-9876543210 : OTP: 482910
Sending PUSH to user_alice via app : You have a new message.
```



Key Insight

Notice the loop: 'for notification in notifications: notification.send()'
The code does not know — or care — whether the object is an EmailNotification, SMSNotification, or PushNotification.
It simply calls .send() on each one, and each object handles it in its own way.
This is the true power of polymorphism — write one piece of code, handle many types.

3.3 Polymorphism with Functions (Duck Typing)

Python also supports a looser form of polymorphism called Duck Typing. The idea is: if an object has the method you need, it does not matter what class it belongs to. Python does not enforce strict type checking.



What is Duck Typing?

"If it walks like a duck and quacks like a duck, it's a duck."

In Python, if an object has a method called `.send()`, you can call `.send()` on it — regardless of what class it is. Python does not check the class hierarchy. It only checks whether the method exists on the object.

Python Code

```
# Duck Typing — No inheritance required

class PDFReport:
    def generate(self):
        print('Generating PDF report...')

class ExcelReport:
    def generate(self):
        print('Generating Excel spreadsheet...')

class HTMLReport:
    def generate(self):
        print('Generating HTML web report...')

# These three classes have NO inheritance relationship at all.
# They just all happen to have a method called generate().

def export_report(report_object):
    report_object.generate() # Works with any object that has generate()

reports = [PDFReport(), ExcelReport(), HTMLReport()]

for report in reports:
    export_report(report)
```

Figure 3.2 — Duck Typing polymorphism

Output

```
Generating PDF report...
Generating Excel spreadsheet...
Generating HTML web report...
```

3.4 Operator Polymorphism — The + Operator

Polymorphism is not only about methods. Python operators like +, -, *, and > behave differently depending on the type of objects they are applied to. This is called Operator Overloading, and it is built into Python.

Python Code

```
# The + operator behaves differently for different types

# With integers — performs addition
print(10 + 20)      # Output: 30

# With strings — performs concatenation
print('Hello' + ' ' + 'World') # Output: Hello World

# With lists — joins the lists
print([1, 2, 3] + [4, 5, 6])  # Output: [1, 2, 3, 4, 5, 6]
```

Figure 3.3 — The + operator behaves differently for different types

Output

```
30
Hello World
[1, 2, 3, 4, 5, 6]
```

This behaviour is not magic — it is because the int, str, and list classes each define their own version of the `__add__()` method (a Special Method, covered in the next chapter). When Python sees `a + b`, it calls `a.__add__(b)`.

3.5 A Complete Example — Document Processing System

Here is a realistic example that combines everything we have learned — inheritance, method overriding, and polymorphism — in a document processing scenario.

Python Code

```
class Document:
    def __init__(self, title, author, content):
        self.title = title
        self.author = author
        self.content = content

    def display_summary(self):
        print('Title :', self.title)
        print('Author :', self.author)
        print('Length :', len(self.content), 'characters')
```

```

def export(self):
    print('Exporting document:', self.title)

class ResearchPaper(Document):
    def __init__(self, title, author, content, journal):
        super().__init__(title, author, content)
        self.journal = journal
    def export(self):
        print('Exporting RESEARCH PAPER to journal:', self.journal, '| Title:', self.title)

class Invoice(Document):
    def __init__(self, title, author, content, invoice_number):
        super().__init__(title, author, content)
        self.invoice_number = invoice_number
    def export(self):
        print('Exporting INVOICE #', self.invoice_number, '| Title:', self.title)

class Newsletter(Document):
    def __init__(self, title, author, content, subscriber_count):
        super().__init__(title, author, content)
        self.subscriber_count = subscriber_count
    def export(self):
        print('Sending NEWSLETTER to', self.subscriber_count, 'subscribers | Title:', self.title)

# Create a list of different document types
documents = [
    ResearchPaper('AI in Healthcare', 'Dr. Mehta', 'Content...' * 100, 'Nature Medicine'),
    Invoice('Q4 Software License', 'Accounts Dept', 'Invoice content...' * 20, 'INV-2025-0041'),
    Newsletter('Monthly Tech Digest', 'Editorial Team', 'News...' * 50, 12500),
]

# Polymorphic behaviour — same loop, different export behaviour
for doc in documents:
    doc.display_summary()
    doc.export()
    print()

```

Figure 3.4 — Polymorphism in a Document Processing System

Output

```

Title : AI in Healthcare
Author : Dr. Mehta
Length : 700 characters
Exporting RESEARCH PAPER to journal: Nature Medicine | Title: AI in Healthcare

Title : Q4 Software License
Author : Accounts Dept
Length : 340 characters

```

Exporting INVOICE # INV-2025-0041 | Title: Q4 Software License

Title : Monthly Tech Digest

Author : Editorial Team

Length : 250 characters

Sending NEWSLETTER to 12500 subscribers | Title: Monthly Tech Digest

ETIHANS
CLOUD & AI ACADEMY



Chapter 4

Special (Magic / Dunder) Methods

Have you wondered how Python "knows" what to do when you write `len(my_list)` or `print(my_object)` or `item1 + item2`? Python does not use special rules for these — instead, every class can define special methods that Python calls automatically when certain actions are performed. These are called Special Methods, also known as Magic Methods or Dunder Methods.

4.1 What are Dunder Methods?

Dunder Methods	Special methods in Python whose names begin and end with double underscores (<code>__</code>). The word 'dunder' stands for 'Double UNDERscore'. Examples: <code>__init__</code> , <code>__str__</code> , <code>__len__</code> , <code>__add__</code> . Python calls these automatically — you do not call them directly.
----------------	---

 **Real-World Analogy — Behind the Scenes Protocols**

When you receive a phone call, you press the green button to answer.
Behind the scenes, your phone follows a set of technical protocols — establish audio, connect signal, etc.
You do not manage those protocols manually — the phone handles them automatically.

Dunder methods are like those behind-the-scenes protocols.
When you write `len(obj)`, Python automatically calls `obj.__len__()`.
When you print an object, Python automatically calls `obj.__str__()`.
You define the protocol; Python triggers it.

4.2 The Most Important Dunder Methods

Dunder Method	When Python Calls It	Example Trigger
<code>__init__(self, ...)</code>	When you create a new object	<code>obj = MyClass()</code>
<code>__str__(self)</code>	When you use <code>print()</code> or <code>str()</code> on the object	<code>print(obj)</code>
<code>__repr__(self)</code>	For developer-friendly representation, used in debugging	<code>repr(obj)</code>
<code>__len__(self)</code>	When you call <code>len()</code> on the object	<code>len(obj)</code>
<code>__add__(self, other)</code>	When you use the <code>+</code> operator	<code>obj1 + obj2</code>

<code>__eq__(self, other)</code>	When you use the <code>==</code> operator	<code>obj1 == obj2</code>
<code>__lt__(self, other)</code>	When you use the <code><</code> operator	<code>obj1 < obj2</code>
<code>__gt__(self, other)</code>	When you use the <code>></code> operator	<code>obj1 > obj2</code>
<code>__contains__(self, item)</code>	When you use the <code>in</code> keyword	<code>'item' in obj</code>
<code>__del__(self)</code>	When the object is about to be destroyed	<code>del obj</code>

4.3 `__init__` — The Constructor

You have already used `__init__` extensively. It is the method Python calls automatically when you create an object from a class. It initializes the object's attributes.

Python Code

```
class Product:
    def __init__(self, name, price, quantity):
        self.name = name
        self.price = price
        self.quantity = quantity
        print('Product created:', self.name) # This runs automatically

# Python calls __init__ automatically:
p1 = Product('Laptop', 75000, 10) # Output: Product created: Laptop
p2 = Product('Mouse', 599, 50) # Output: Product created: Mouse
```

Figure 4.1 — The `__init__` constructor method

Output

```
Product created: Laptop
Product created: Mouse
```

4.4 `__str__` — Human-Readable String Representation

By default, if you print an object, Python displays something like `<__main__.Product object at 0x7f4b...>` — which is not useful. By defining `__str__`, you control exactly what appears when the object is printed.

Python Code

```
class Product:
    def __init__(self, name, price, quantity):
```

```
self.name = name
self.price = price
self.quantity = quantity

def __str__(self):    # Defines what print() shows
    return f'Product: {self.name} | Price: Rs.{self.price} | Stock: {self.quantity}'

p1 = Product('Laptop', 75000, 10)
p2 = Product('Mouse', 599, 50)

print(p1)    # Calls p1.__str__() automatically
print(p2)    # Calls p2.__str__() automatically
print(str(p1)) # Also calls __str__()
```

*Figure 4.2 — The __str__ method***Output**

```
Product: Laptop | Price: Rs.75000 | Stock: 10
Product: Mouse | Price: Rs.599 | Stock: 50
Product: Laptop | Price: Rs.75000 | Stock: 10
```

4.5 `__len__` — Defining Length

The `__len__` method lets you define what `len()` returns for your objects. This is particularly useful for classes that represent a collection of items.

Python Code

```
class ShoppingCart:
    def __init__(self, customer_name):
        self.customer_name = customer_name
        self.items = []

    def add_item(self, product_name, price):
        self.items.append({'product': product_name, 'price': price})
        print(product_name, 'added to cart.')

    def __len__(self):    # Defines len(cart)
        return len(self.items)

    def __str__(self):
        return f"Cart of {self.customer_name} — {len(self.items)} item(s)"

cart = ShoppingCart('Alice')
cart.add_item('Laptop', 75000)
```

```

cart.add_item('Mouse', 599)
cart.add_item('Keyboard', 1299)

print(cart)      # Calls __str__
print('Items in cart:', len(cart)) # Calls __len__

```

Figure 4.3 — The `__len__` method on a shopping cart

Output

```

Laptop added to cart.
Mouse added to cart.
Keyboard added to cart.
Cart of Alice — 3 item(s)
Items in cart: 3

```

4.6 `__add__` — Defining the + Operator

The `__add__` method allows you to define what happens when the `+` operator is used between two objects of your class.

Python Code

```

class Budget:
    def __init__(self, department, amount):
        self.department = department
        self.amount = amount

    def __add__(self, other): # Defines Budget + Budget
        combined_dept = self.department + ' & ' + other.department
        combined_amount = self.amount + other.amount
        return Budget(combined_dept, combined_amount)

    def __str__(self):
        return f'Department: {self.department} | Budget: Rs.{self.amount:,}'

marketing = Budget('Marketing', 500000)
engineering = Budget('Engineering', 1200000)

print(marketing)
print(engineering)

combined = marketing + engineering # Calls marketing.__add__(engineering)
print()
print('Combined Budget:')
print(combined)

```

Figure 4.4 — The `__add__` method**Output**

Department: Marketing | Budget: Rs.500,000

Department: Engineering | Budget: Rs.1,200,000

Combined Budget:

Department: Marketing & Engineering | Budget: Rs.1,700,000

4.7 `__eq__` — Defining Equality

By default, `==` checks whether two variables point to the same object in memory. By defining `__eq__`, you can specify what it means for two objects to be considered equal based on their content.

Python Code

```
class Employee:
    def __init__(self, name, employee_id):
        self.name = name
        self.employee_id = employee_id

    def __eq__(self, other):
        # Two employees are equal if they have the same employee_id
        return self.employee_id == other.employee_id

    def __str__(self):
        return f'{self.name} (ID: {self.employee_id})'

emp1 = Employee('Ravi Mehta', 'EMP001')
emp2 = Employee('Ravi Mehta', 'EMP001') # Same ID
emp3 = Employee('Priya Singh', 'EMP002') # Different ID

print(emp1 == emp2) # True — same employee_id
print(emp1 == emp3) # False — different employee_id
```

Figure 4.5 — The `__eq__` method**Output**

True

False

4.8 A Complete Example — Student Record System

Let us combine multiple dunder methods into one cohesive, professional class that demonstrates how they all work together.

Python Code

```
class Student:
    def __init__(self, name, student_id, marks_list):
        self.name = name
        self.student_id = student_id
        self.marks_list = marks_list

    def __str__(self):
        avg = sum(self.marks_list) / len(self.marks_list)
        return (f'Student: {self.name} | ID: {self.student_id} | '
                f'Average Marks: {avg:.1f}')

    def __len__(self):
        return len(self.marks_list) # Number of subjects

    def __eq__(self, other):
        return self.student_id == other.student_id

    def __gt__(self, other):
        return sum(self.marks_list) > sum(other.marks_list)

    def __lt__(self, other):
        return sum(self.marks_list) < sum(other.marks_list)

# Create student objects
s1 = Student('Alice Sharma', 'S2025001', [88, 92, 76, 95, 89])
s2 = Student('Bob Patel', 'S2025002', [72, 80, 85, 78, 90])
s3 = Student('Alice Sharma', 'S2025001', [88, 92, 76, 95, 89]) # Same as s1

print(s1)          # __str__
print(s2)          # __str__
print()
print('Subjects in s1 record:', len(s1)) # __len__
print('s1 == s3 (same ID):', s1 == s3)   # __eq__
print('s1 > s2 (higher total):', s1 > s2) # __gt__
print('s2 < s1 (lower total):', s2 < s1) # __lt__
```

Figure 4.6 — Combining multiple dunder methods in a Student class

Output

Student: Alice Sharma | ID: S2025001 | Average Marks: 88.0

Student: Bob Patel | ID: S2025002 | Average Marks: 81.0

Subjects in s1 record: 5

```
s1 == s3 (same ID): True  
s1 > s2 (higher total): True  
s2 < s1 (lower total): True
```

ETIHANS
CLOUD & AI ACADEMY



Chapter 5

Exception Handlingusing try and except

Even perfectly written programs encounter unexpected situations — a file that does not exist, a user who enters letters instead of numbers, a network connection that drops. When something goes wrong during a program's execution, Python raises an Exception. Without exception handling, the program crashes. With it, you can gracefully respond to errors and keep your program running.

5.1 What is an Exception?

Exception	An error that occurs during the execution of a program, disrupting the normal flow of instructions. When Python encounters something it cannot handle — like dividing by zero or accessing a file that does not exist — it raises (throws) an Exception.
-----------	--

 **Real-World Analogy — ATM Machine**

An ATM machine is programmed to dispense cash.
But what if the card is expired? What if the PIN is wrong? What if funds are insufficient?

A well-designed ATM does not crash — it shows a specific error message and asks you what to do next. Exception handling is how programmers build this kind of resilient, error-aware software. Instead of crashing, the program recognizes the problem and responds appropriately.

5.2 What Happens Without Exception Handling?

Let us see what happens when an error occurs in a program that has no exception handling.

Python Code

```
# Program without exception handling
withdrawal_amount = int(input('Enter withdrawal amount: '))
account_balance = 5000
remaining = account_balance / withdrawal_amount
print('Remaining balance:', remaining)
```

Figure 5.1 — A program with no exception handling

If the user types 0 as the withdrawal amount, Python raises an error and the program crashes immediately:

Error Output

ZeroDivisionError: division by zero
Traceback (most recent call last):
 File 'atm.py', line 4, in <module>
 remaining = account_balance / withdrawal_amount

The user sees a technical error and the program stops — a poor experience for any real application. Exception handling prevents this.

5.3 Common Python Exceptions

Exception Name	When It Occurs	Example
ZeroDivisionError	Dividing a number by zero	10 / 0
ValueError	Wrong type of value passed to a function	int('hello')
TypeError	Operation on incompatible types	'text' + 5
FileNotFoundError	Trying to open a file that does not exist	open('missing.txt')
IndexError	Accessing a list index that does not exist	my_list[100]
KeyError	Accessing a dictionary key that does not exist	d['unknown_key']
NameError	Using a variable that has not been defined	print(undefined_var)
AttributeError	Accessing an attribute that does not exist on an object	None.upper()

5.4 The try...except Block

try...except	A Python construct that allows you to try running a block of code, and if an exception occurs, handle it gracefully in the except block instead of crashing.
--------------	--

Basic Structure

Python Code
Basic try...except structure

```
try:
    # Code that might cause an error
    risky_code_here

except ExceptionType:
    # Code that runs IF the exception occurs
    handle_the_error
```

Figure 5.2 — Basic try...except syntax

How try...except Executes

1. Python enters the try block and runs the code.
- 2a. If NO error occurs → Python skips the except block entirely.
- 2b. If an ERROR occurs → Python immediately jumps to the except block.
3. After the except block, the program continues normally.

The program does NOT crash — it handles the error and moves on.

Example — Fixing the Division Problem

Python Code

```
# With exception handling — graceful error management

account_balance = 5000

try:
    withdrawal_amount = int(input('Enter withdrawal amount: '))
    remaining = account_balance / withdrawal_amount
    print('Transaction successful. Remaining balance: Rs.', remaining)

except ZeroDivisionError:
    print('Error: Withdrawal amount cannot be zero. Please enter a valid amount.')

except ValueError:
    print('Error: Please enter a numeric value only.')

print('Thank you for using our ATM.')
```

Figure 5.3 — Exception handling in an ATM-style program

Now, regardless of what the user enters, the program handles it gracefully and the final message always prints:

Output

```
# If user enters 0:
Error: Withdrawal amount cannot be zero. Please enter a valid amount.
Thank you for using our ATM.

# If user enters 'abc':
Error: Please enter a numeric value only.
Thank you for using our ATM.

# If user enters 1000:
Transaction successful. Remaining balance: Rs. 5.0
Thank you for using our ATM.
```

5.5 The else and finally Clauses

The try...except construct can also include two optional sections: else and finally, each serving a distinct purpose.

Clause	When It Runs	Purpose
try	Always — first	Code that might raise an exception
except	Only if an exception occurs in try	Handle the specific error
else	Only if NO exception occurred in try	Code to run when everything went well
finally	Always — whether or not an error occurred	Cleanup code (close files, release resources)

Python Code

```
# try...except...else...finally — complete structure

def process_student_record(student_id, marks_list):
    try:
        average = sum(marks_list) / len(marks_list)
        grade = 100 / average      # Intentional division to show flow

    except ZeroDivisionError:
        print('Error: Cannot compute average — marks list is empty.')

    else:
        # Runs ONLY if no exception occurred
        print('Record processed successfully.')
        print('Student ID :', student_id)
        print('Average  :', round(average, 2))
```

```
finally:
    # ALWAYS runs — used for cleanup
    print('Processing complete for student:', student_id)
    print('---')

process_student_record('S001', [88, 92, 76, 85])
process_student_record('S002', [])      # Empty list — triggers error
```

Figure 5.4 — The full try...except...else...finally structure

Output

```
Record processed successfully.
Student ID : S001
Average   : 85.25
Processing complete for student: S001
---
Error: Cannot compute average — marks list is empty.
Processing complete for student: S002
---
```

Key Observation

Notice that 'Processing complete for student: S002' still prints — even though an error occurred. The finally block ALWAYS runs. This is essential for releasing resources like:

- Closing a database connection
- Closing an open file
- Releasing a network socket

Without finally, these resources might be left open if an error occurs.

5.6 Handling Multiple Exceptions

A single try block can have multiple except clauses — one for each type of error you expect. Python will match the first except clause that matches the raised exception.

Python Code

```
def load_student_data(file_name, index):
    try:
        with open(file_name, 'r') as f:
            data = f.readlines()
            record = data[index]
            marks = int(record.strip())
            print('Marks loaded:', marks)
```

```
except FileNotFoundError:
    print('Error: The file', file_name, 'was not found.')

except IndexError:
    print('Error: Record at index', index, 'does not exist.')

except ValueError:
    print('Error: The data is not a valid number.')

except Exception as e:
    # Catch-all for any other unexpected error
    print('An unexpected error occurred:', e)

finally:
    print('Data loading attempt complete.')

load_student_data('records.txt', 2) # File does not exist
load_student_data('marks.csv', 999) # Index does not exist
```

*Figure 5.5 — Handling multiple exceptions***Output**

Error: The file records.txt was not found.
Data loading attempt complete.

Error: The file marks.csv was not found.
Data loading attempt complete.

**Best Practice — The except Exception Catch-All**

`except Exception as e` is a catch-all that captures any exception not caught by earlier `except` clauses. The variable `e` contains the error message, which you can print or log. Always place this as the LAST `except` clause — specific exceptions should come first.

Good practice: Log the error to a file in production systems rather than printing it to the screen.

5.7 Raising Exceptions Manually — raise

Sometimes, you want to raise an exception yourself — when a value that was passed to a function is invalid. You use the `raise` keyword to trigger an exception intentionally.

Python Code

```
class Employee:
```



```
def __init__(self, name, salary):
    if salary < 0:
        raise ValueError('Salary cannot be negative. Received: ' + str(salary))
    if not name:
        raise ValueError('Employee name cannot be empty.')
    self.name = name
    self.salary = salary

def __str__(self):
    return f'{self.name} — Salary: Rs.{self.salary:,}'

# Valid employee
try:
    emp1 = Employee('Ravi Mehta', 45000)
    print(emp1)
except ValueError as e:
    print('Validation Error:', e)

# Invalid salary
try:
    emp2 = Employee('Priya Singh', -5000)
    print(emp2)
except ValueError as e:
    print('Validation Error:', e)

# Empty name
try:
    emp3 = Employee("", 30000)
    print(emp3)
except ValueError as e:
    print('Validation Error:', e)
```

Figure 5.6 — Raising exceptions manually with raise

Output

```
Ravi Mehta — Salary: Rs.45,000
Validation Error: Salary cannot be negative. Received: -5000
Validation Error: Employee name cannot be empty.
```

5.8 A Complete Professional Example — File Processing System

Let us build a realistic file processing utility that combines all exception handling concepts covered in this chapter.

Python Code

```
def read_sales_report(file_path):  
    """  
    Reads a sales report CSV file and calculates total and average sales.  
    Handles all common file and data errors gracefully.  
    """  
  
    total_sales = 0  
    record_count = 0  
  
    try:  
        file = open(file_path, 'r')  
        print('File opened successfully:', file_path)  
  
        for line_number, line in enumerate(file, start=1):  
            line = line.strip()  
            if not line:  
                continue # Skip empty lines  
  
            try:  
                sales_value = float(line)  
                total_sales += sales_value  
                record_count += 1  
            except ValueError:  
                print(f' Warning: Line {line_number} has invalid data: "{line}" — Skipped.')  
  
    except FileNotFoundError:  
        print('Error: File not found —', file_path)  
        print('Please verify the file path and try again.')  
        return None  
  
    except PermissionError:  
        print('Error: No permission to read file —', file_path)  
        return None  
  
    else:  
        if record_count > 0:  
            average = total_sales / record_count  
            print(f'\nReport Summary:')  
            print(f' Total Records : {record_count}')  
            print(f' Total Sales   : Rs.{total_sales:,.2f}')  
            print(f' Average Sales : Rs.{average:,.2f}')  
            return total_sales  
        else:  
            print('Warning: No valid records found in file.')  
            return 0  
  
    finally:  
        try:  
            file.close()  
            print('\nFile closed successfully.')  
        except NameError:
```

```

        pass # File was never opened — nothing to close

# Test the function
read_sales_report('sales_q4.csv') # Likely does not exist in test
read_sales_report('nonexistent_file.txt') # Definitely does not exist

```

Figure 5.7 — Complete exception handling in a file processing system

Output

Error: File not found — sales_q4.csv
Please verify the file path and try again.

Error: File not found — nonexistent_file.txt
Please verify the file path and try again.

5.9 Exception Handling with OOP — Combining Concepts

Exception handling becomes even more powerful when combined with the OOP concepts from earlier chapters. Let us see how Inheritance, Method Overriding, and Exception Handling work together in a real system.

Python Code

```

# Exception Handling + OOP — Inventory Management System

class InventoryItem:
    def __init__(self, item_name, item_code, quantity, unit_price):
        if quantity < 0:
            raise ValueError(f'Quantity cannot be negative for {item_name}')
        if unit_price <= 0:
            raise ValueError(f'Unit price must be positive for {item_name}')
        self.item_name = item_name
        self.item_code = item_code
        self.quantity = quantity
        self.unit_price = unit_price

    def total_value(self):
        return self.quantity * self.unit_price

    def __str__(self):
        return (f'{self.item_name} ({self.item_code}) | '
                f'Qty: {self.quantity} | Price: Rs.{self.unit_price} | '
                f'Total: Rs.{self.total_value():,}')

class PerishableItem(InventoryItem):
    def __init__(self, item_name, item_code, quantity, unit_price, expiry_days):
        super().__init__(item_name, item_code, quantity, unit_price)

```

```

        if expiry_days <= 0:
            raise ValueError('Expiry days must be greater than zero.')
        self.expiry_days = expiry_days

    def __str__(self):
        base = super().__str__()
        return base + f' | Expires in: {self.expiry_days} days'

# Testing with exception handling
items_to_create = [
    ('Laptop', 'TECH001', 50, 75000, None),
    ('Milk Carton', 'FOOD001', 200, 45, 5),
    ('Mouse', 'TECH002', -10, 599, None), # Invalid quantity
    ('Bread', 'FOOD002', 100, 35, -1), # Invalid expiry
]

inventory = []

for name, code, qty, price, expiry in items_to_create:
    try:
        if expiry is not None:
            item = PerishableItem(name, code, qty, price, expiry)
        else:
            item = InventoryItem(name, code, qty, price)
        inventory.append(item)
        print('Added:', item)
    except ValueError as e:
        print('Skipped', name, '— Validation Error:', e)

print()
print('Final Inventory (' + str(len(inventory)) + ' items):')
for item in inventory:
    print('•', item)

```

Figure 5.8 — OOP + Exception Handling in an Inventory Management System

Output

Added: Laptop (TECH001) | Qty: 50 | Price: Rs.75000 | Total: Rs.3,750,000
 Added: Milk Carton (FOOD001) | Qty: 200 | Price: Rs.45 | Total: Rs.9,000 | Expires in: 5 days
 Skipped Mouse — Validation Error: Quantity cannot be negative for Mouse
 Skipped Bread — Validation Error: Expiry days must be greater than zero.

Final Inventory (2 items):

- Laptop (TECH001) | Qty: 50 | Price: Rs.75000 | Total: Rs.3,750,000
- Milk Carton (FOOD001) | Qty: 200 | Price: Rs.45 | Total: Rs.9,000 | Expires in: 5 days

Unit Summary

In this unit, you have covered five core pillars of Object-Oriented Programming in Python. Here is a consolidated recap of everything covered:

Chapter	Topic	Core Concept
1	Inheritance	A child class automatically gets all attributes and methods of a parent class, enabling code reuse and organized class hierarchies.
2	Method Overriding	A child class can redefine a parent's method with its own implementation. <code>super()</code> lets you extend the parent's behaviour.
3	Polymorphism	Different objects can respond to the same method call in their own way. Write one interface, handle many types of objects.
4	Special Methods	Dunder methods (<code>__str__</code> , <code>__len__</code> , <code>__add__</code> , etc.) let you define how Python's built-in operations behave on your custom objects.
5	Exception Handling	<code>try...except...else...finally</code> lets programs handle runtime errors gracefully instead of crashing, producing robust, production-ready software.



Connecting the Concepts

These five concepts are not isolated — they work together in every real Python application:

- Inheritance creates class hierarchies (Employee → Manager → SeniorManager)
- Method Overriding lets each class in the hierarchy customize its behaviour
- Polymorphism lets you write one loop that works with all types in the hierarchy
- Dunder Methods make your objects behave like native Python types
- Exception Handling makes your program robust against invalid data and runtime errors

Master these five concepts and you will have a solid foundation for building professional, maintainable Python applications.